An agent in your Angular app

CopilotKit, and then the same demo again in C#

Satellit · internal talk

That was two lines of Angular

```
// app/src/app/app.config.ts
providers: [
  provideCopilotKit({ runtimeUrl: 'http://localhost:8200/api/copilotkit' }),
]
```

```
<!-- app/src/app/app.html -->
<copilot-chat />
```

A provider and a component. There is no third thing yet: no state, no tools, nothing that has heard of the Board you can see next to the chat.

Where this is going

```
Phase 1    Angular  ⟶  Node runtime  ⟶  OpenAI
Phase 2    Angular  ⟶  C# agent      ⟶  OpenAI
```

Same app. Same Board. Same five tools. **One line of Angular changes between them.**

Phase 2 is where most of you live, so it gets a real read-through, not an epilogue.

Trace 1 — what actually went over the wire

```
{
  "model": "gpt-5-mini",
  "input": [
    { "role": "developer",
      "content": "You are a helpful assistant.\nKeep replies short and plain: they are read off a projector." },
    { "role": "user",
      "content": [{ "type": "input_text",
                    "text": "What usually goes wrong when a company builds its own employee onboarding portal?" }] }
  ]
}
```

```
Common failures and how to avoid them
- Scope creep: project balloons with extra features …
- Integration gaps: payroll, HRIS, SSO, benefits, devices don't connect …
```

Captured from the runtime with the frontend of the previous slide behind it. The `tools` array is cut from the paste: the only two in it belong to the Team directory, which the runtime attaches to every turn, and slide 13 comes back to them.

So what is an agent?

- **A model.** Trace 1 has one, and that is all it has.
- **Your app's context.** What the model is allowed to know. Trace 1 has none.
- **Tools.** What the model is allowed to ask for. Nothing in trace 1 can touch the Board.
- **A loop.** Call the model, run what it asked for, feed the result back, call it again.

Everything between here and the C# is those four things. The rest is plumbing.

Trace 2 — the same turn, with a Board and a verb

```
{ "role": "developer",
  "content": "You are a helpful assistant. …\n
              ## Context from the application\n
              The current task board:\n
              [{\"id\":\"T-4\",\"title\":\"Build the profile page\",\"status\":\"doing\",\"assignee\":\"Amira\"}, …]" }
```

```
{ "role": "user", "content": [{ "type": "input_text", "text": "Amira finished the profile page — mark it done." }] }

{ "type": "function_call", "call_id": "call_wYvh…", "name": "moveTask",
  "arguments": "{\"id\":\"T-4\",\"status\":\"done\"}" }
```

```
{ "type": "function_call_output", "call_id": "call_wYvh…",
  "output": "Moved T-4 “Build the profile page” to done." }
```

```
Done — I moved "Build the profile page" to done.
```

Tool-calling

- A tool is **a name, a description, and a JSON schema**. That is the whole declaration.
- The model **requests** a call. **Your code runs it.** The loop **feeds the result back**.
- The description is not documentation. It is the only thing steering the choice.
- The loop needs a step budget: `maxSteps: 5`.

The runtime default is **1** — enough for the model to ask and never hear the answer. Beat 6 chains two calls in one turn and needs at least 3.

CopilotKit

- **A chat UI.** `<copilot-chat>`, and a popup and a sidebar you are not seeing today.
- **A runtime.** The Node tier on 8200. It holds the model key and the loop.
- **A protocol.** AG-UI, between the browser and whatever is answering. Defined on slide 14, because that is where it starts to matter.

`@copilotkit/angular` **0.3.1** — a first-party, signal-based port of the React package. MIT. Two months old, twenty-five versions, so the version is pinned exactly and so is everything else in this repo.

The agent reads the Board

```
connectAgentContext(() => ({
  description: 'The current task board',
  value: JSON.stringify(this.board.tasks()),
}));
```

- **An accessor, not an object.** It wraps an `effect()`, so a plain object registers once and never follows the signal again.
- One entry, re-serialised on every turn. That is the entire read channel.
- There is also `injectAgentStore()`, shared state both ways. Not used here: the Board stays owned by Angular signals, one writer, nothing to desync on stage.

The agent changes the Board

```
registerFrontendTool({
  name: 'moveTask',
  description: `Move a Task to a different status. ${BY_ID}`,
  parameters: z.object({ id: ID, status: z.enum(STATUSES) }),
  handler: async ({ id, status }) ⇒ board.moveTask(id, status),
  component: ToolOutcome,
});
```

```
registerHumanInTheLoop({
  name: 'deleteTask',
  description: `Remove a Task from the board for good. … the app puts the confirm dialog in
    front of them and tells you whether they went through with it, so never ask for
    confirmation yourself. ${BY_ID}`,
  parameters: z.object({ id: ID }),
  component: DeleteConfirm,   // no handler: the run parks until the user clicks
});
```

Five tools, all registered in **Angular**. The Node tier's own `tools: []` is empty.

It resolves your words, and it can resolve them wrong

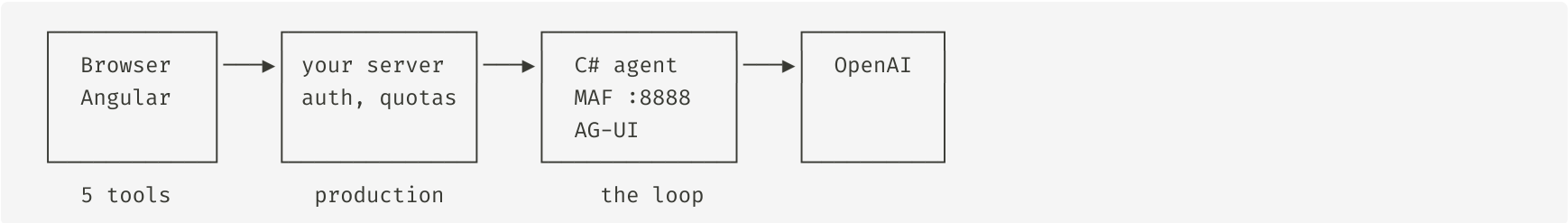
A tool can return UI

```
registerFrontendTool({
  name: 'showBoard',
  description: 'Show the user the whole board. … it renders the three columns in the chat and changes nothing.',
  parameters: z.object({}),
  handler: async () ⇒ 'The board is on screen in the chat.',
  component: MiniBoard,
});
```

- `component:` is Angular's spelling of React's `render`. A component, not a function.
- A **mutating tool** changes the Board: create, move, assign, delete.
- A **rendering tool** changes nothing. `showBoard` exists only to put something on screen.

Reaching outside the app: MCP

Architecture



AG-UI is the protocol in every one of those arrows: a stream of typed events — `RUN_STARTED`, `TEXT_MESSAGE_CONTENT`, `TOOL_CALL_START`. The five tools ride it as a plain `tools` field, which is why they survive the middle box changing.

MCP does not disappear with the Node tier. **It moves** — MAF has first-class MCP support in .NET.

Microsoft's guidance is explicit: **do not expose an AG-UI server straight to a browser.** A client that can POST arbitrary input can inject system messages and forge tool results. For a demo on a laptop this is the right shape. For production, the box comes back.

The diff

```
+import { HttpAgent } from '@ag-ui/client';
 import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
 import { provideCopilotKit } from '@copilotkit/angular';

 export const appConfig: ApplicationConfig = {
   providers: [
     provideBrowserGlobalErrorListeners(),
-    // Absolute, and cross-origin on purpose. No dev-server proxy: phase 2 swaps this one
-    // line for an agent on another origin, so a proxy would be scaffolding to delete.
-    provideCopilotKit({ runtimeUrl: 'http://localhost:8200/api/copilotkit' }),
+    // The whole diff against `main`. The browser now talks straight to the C# agent on 8888,
+    // with no Node tier in the path — which is why `main` never used a dev-server proxy.
+    provideCopilotKit({
+      selfManagedAgents: { default: new HttpAgent({ url: 'http://localhost:8888/' }) },
+    }),
   ],
 };
```

One file, six lines added and three taken out. `git diff main phase-2` is `app/src/app.config.ts` and nothing else.

agent/agent.cs

```
#:sdk Microsoft.NET.Sdk.Web
#:package Microsoft.Agents.AI.Hosting.AGUI.AspNetCore@1.19.0-preview.260822.1
#:package Microsoft.Agents.AI.OpenAI@1.19.0
#:package DotNetEnv@3.2.0

// … eight using lines, every one of them transitive
Env.Load("../.env");        // … and a one-line exit if the file is not there
var builder = WebApplication.CreateBuilder(args);
builder.WebHost.UseUrls("http://localhost:8888");
builder.Services.AddAGUIServer();
builder.Services.AddCors(o => o.AddDefaultPolicy(
    p => p.AllowAnyOrigin().AllowAnyHeader().AllowAnyMethod()));

var chatClient = new OpenAIClient(builder.Configuration["OPENAI_API_KEY"])
    .GetChatClient("gpt-5-mini").AsIChatClient();  // … and one for a missing key
AIAgent agent = chatClient
    .AsAIAgent(
        name: "BoardAgent",
        instructions: "You are a helpful assistant.\n"
            + "Keep replies short and plain: they are read off a projector.")
    .AsBuilder()
    .Use(async (messages, session, options, next, ct) =>
    {
        //  ← slide 17: fold `context` back into the messages
        RewrapFrontendTools(options);
        await next(messages, session, options, ct);
    })
    .Build();

var app = builder.Build();
app.UseCors();
app.MapAGUIServer("/", agent);
await app.RunAsync();
```

The eleven lines the Node tier was doing for you

```
.Use(async (messages, session, options, next, ct) =>
{
    if (options is ChatClientAgentRunOptions { ChatOptions: { } chatOptions }
        && chatOptions.TryGetRunAgentInput(out RunAgentInput? input)
        && input.Context is { Count: > 0 } entries)
    {
        var text = string.Join("\n\n", entries.Select(e => $"{e.Description}:\n{e.Value}"));
        messages = [new ChatMessage(ChatRole.System, text), .. messages];
    }

    await next(messages, session, options, ct);
})
```

`MapAGUIServer` reads `messages`, `tools` and `resume`, and **drops** `context`. Without this, "mark it done" has no `T-4` to resolve.

Phase 1 was never doing nothing here. `BuiltInAgent` folds the same context into the same system message on every turn, invisibly, inside a dependency.

Taking this further

- **Backend tools in C#**: `AIFunctionFactory.Create(MyMethod)`, one line. Left out here on purpose — a C# tool would make phase 2 behave differently from phase 1, and sameness was the whole claim.
- **The key**: `dotnet user-secrets` rather than a root `.env`. The `.env` is a demo convenience and nothing else.
- **MCP into MAF**: the directory moves into the C# agent rather than disappearing.

Licensing

`selfManagedAgents` — the phase-2 prop — is documented as an Enterprise-tier feature. **The gate is a commercial term, not a code path**: every package is MIT, the check fails open, there is no licence server and no phone-home. An unlicensed machine hits zero limits. The obligation for production use is real regardless.

Resources

This repo
`github.com/FabienDehopre-Satellit/copilotkit-demo`

CopilotKit for Angular
`docs.copilotkit.ai/angular`

Microsoft's AG-UI sample
`github.com/microsoft/agent-framework`
`dotnet/samples/02-agents/AGUI`

`dotnet run app.cs`
`devblogs.microsoft.com/dotnet`
`announcing-dotnet-run-app/`